LLM ENGINEERING

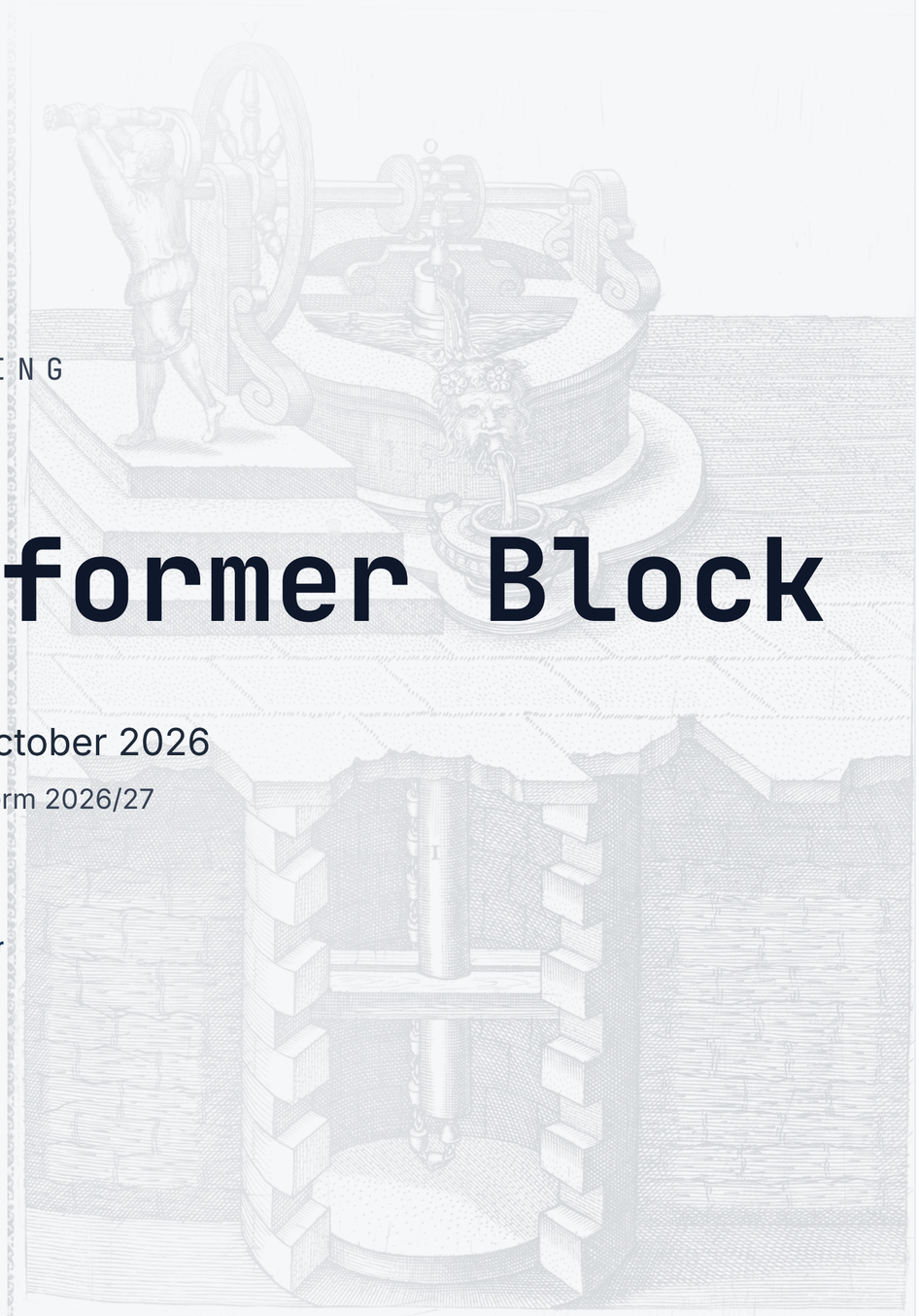
Week 02

Building a Transformer Block

Bethgelab · Thursday, 22 October 2026

University of Tübingen · Winter term 2026/27

Tübingen AI Center



Announcements

Next week, 29 October

- Lecture as a video, not in person
- Tutorials as usual

Quiz 1 on ILIAS

- Weeks 1, 2 and Bonus 1
- Due Friday 30 October, end of day

First exercise: this lecture

- Due Wednesday 28 October, 22:00
- Email me: partner and GitHub names

You have the objective, not yet the function

$$P(x_1, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_{<t}) \tag{1}$$

- Raw window: `[B,S+1]` token IDs
- Shift once into inputs and labels, both `[B,S]`
- Wanted: logits `[B,S,V]` for one distribution per position

How does a model predict the next token, and at what cost?

- **Attention**: each position looks at itself and earlier tokens
- Block: residuals, normalization, multilayer perceptron (MLP)
- Cost: parameters, floating-point operations (FLOPs), memory
- Exercise: your first trained language model



What must a function do?

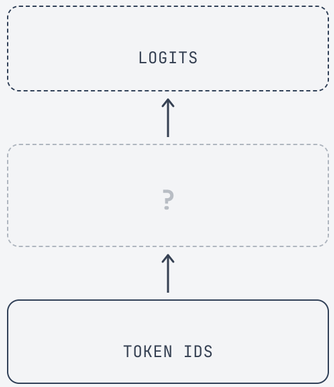
Input: the token IDs of “The cat sat on the”

Output: a distribution over the next token



Today's build, one diagram

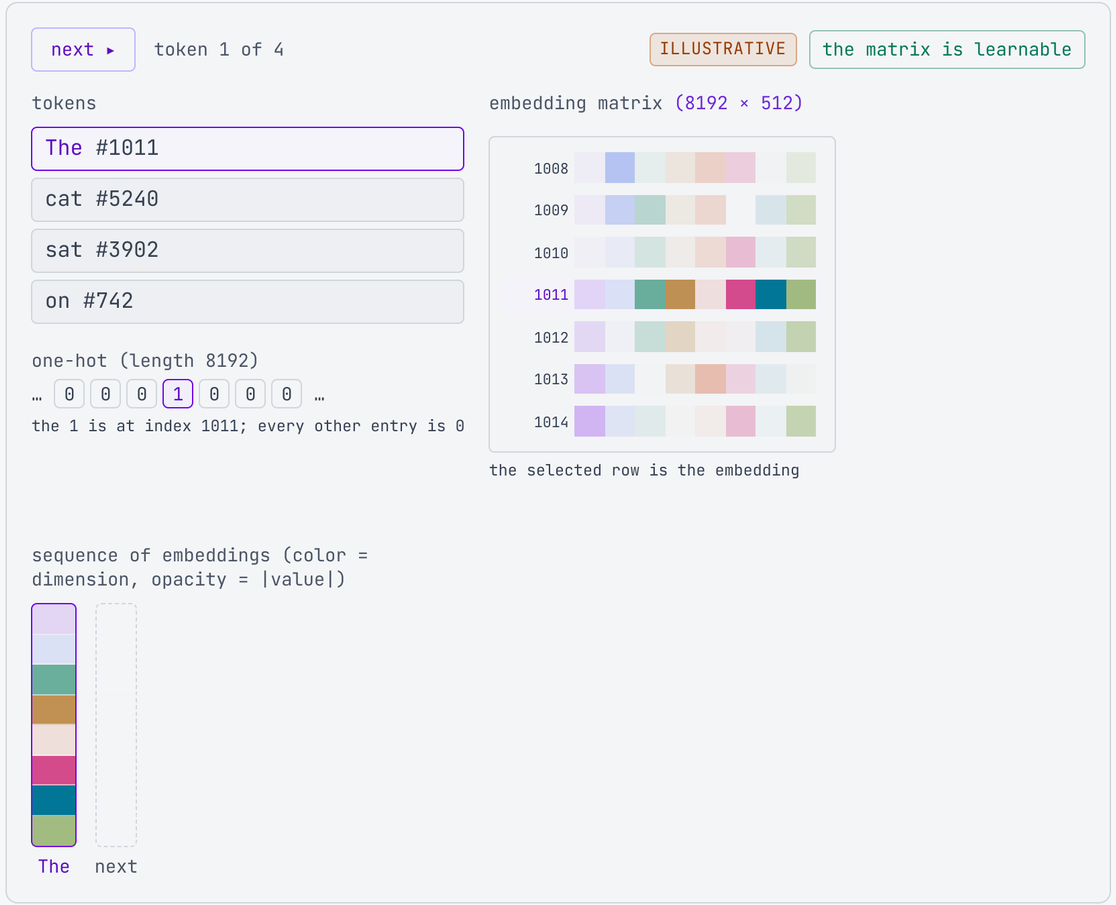
- Token embeddings
- Position signal
- Causal self-attention
- MLP
- Output head



An embedding table turns IDs into vectors

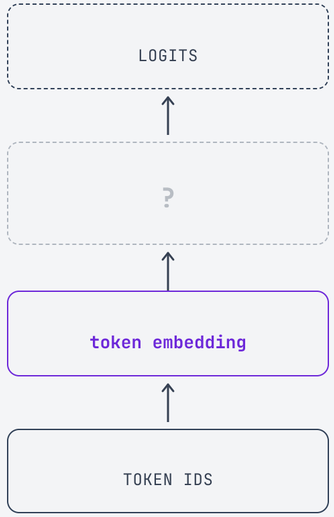
(2) $$E \in \mathbb{R}^{V \times D}, \qquad [B, S] \longrightarrow [B, S, D]$$

- IDs are addresses, not magnitudes
- IDs of shape `[B,S]` select rows, giving `[B,S,D]`
- Table size: `VD = 8192·512 = 4,194,304` parameters



Add embeddings to the model

- Each ID becomes one `D`-wide vector
- The same table returns as the output head



Attention is a soft, learned lookup

- Each position emits a **query**
- Every position offers a **key** and **value**
- The query's scores against the keys weight the values



Scaled dot-product attention

Projection	Weights	Result
`Q=XW_Q`	`W_Q:[D,d_k]`	`Q:[B,S,d_k]`
`K=XW_K`	`W_K:[D,d_k]`	`K:[B,S,d_k]`
`V=XW_V`	`W_V:[D,d_k]`	`V:[B,S,d_k]`

(3)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

output projection

$$O = HW_O, \qquad W_O : [d_k, D], \qquad O : [B, S, D]$$

- Scores and probabilities have shape `[B,S,S]`
- Softmax across keys, per query
- The course uses one head as wide as the model, `d_k = D`

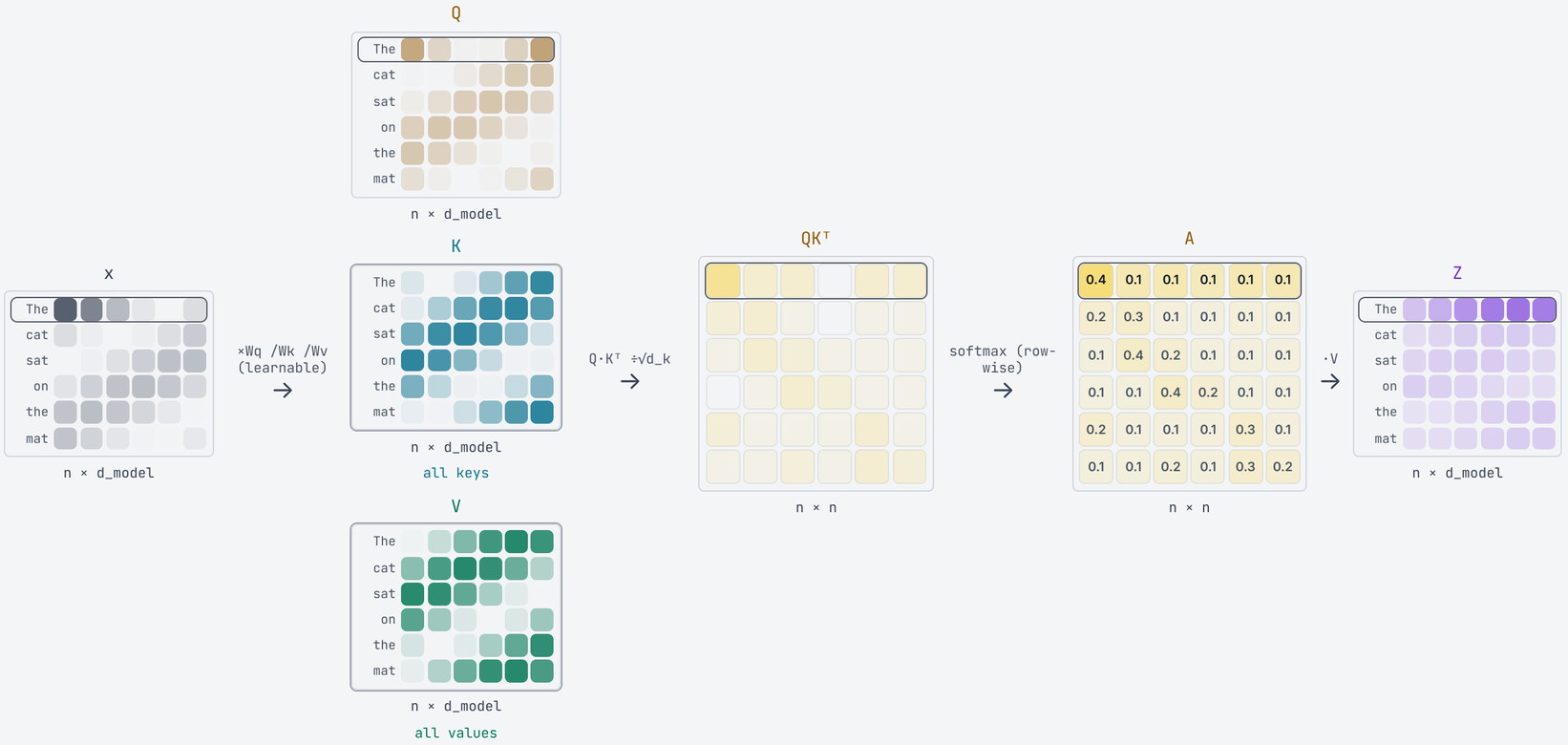
Will rows get sharper or flatter?

Turn off the √d_k scaling.

trace token: The | cat | sat | on | the | mat

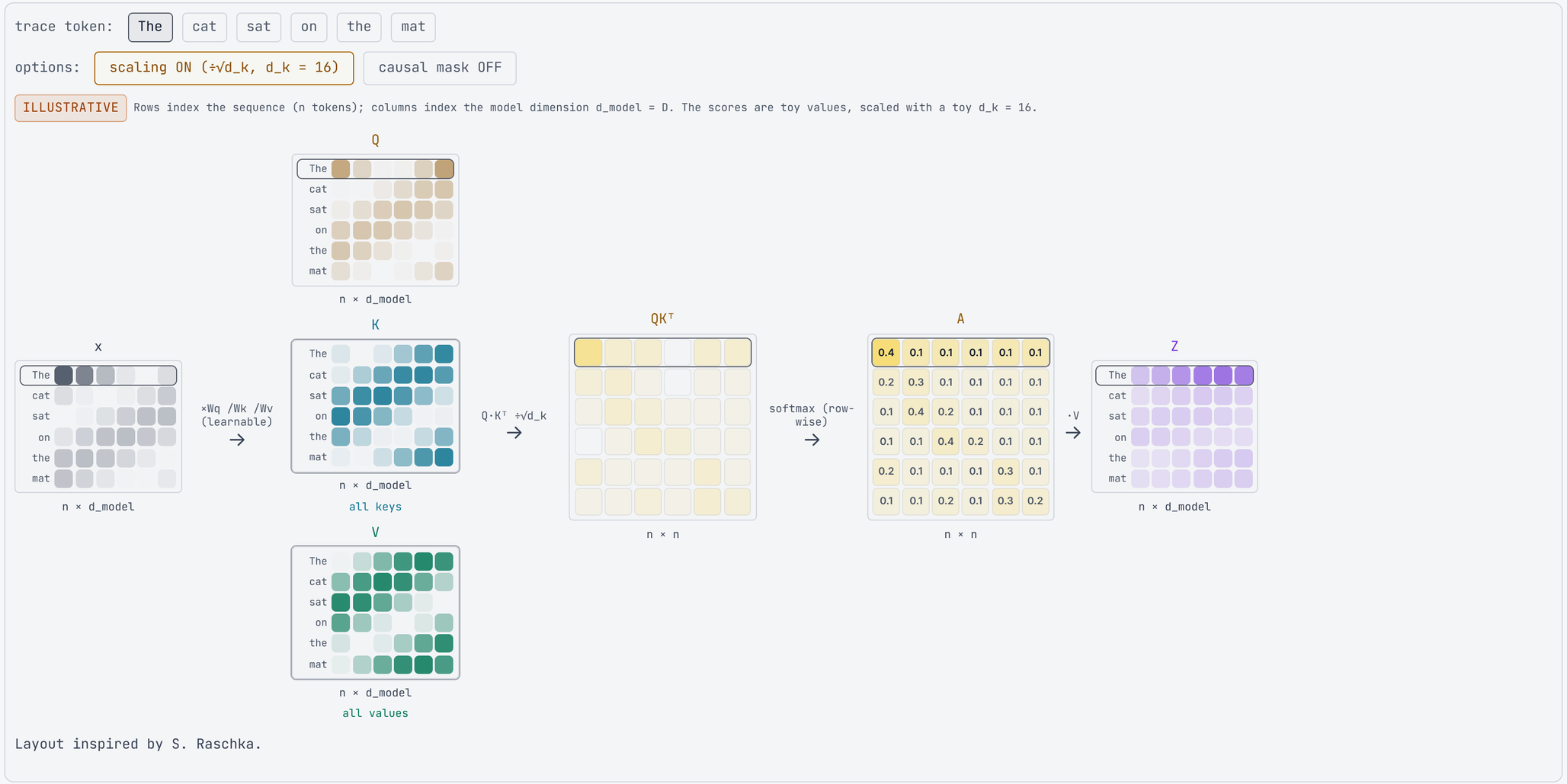
options: scaling ON (÷√d_k, d_k = 16) | causal mask OFF

ILLUSTRATIVE Rows index the sequence (n tokens); columns index the model dimension d_model = D. The scores are toy values, scaled with a toy d_k = 16.



Layout inspired by S. Raschka.

Without scaling, attention rows sharpen



- Sharper: one key takes most weight
- Unscaled scores are 4 times larger at `d_k = 16`

Why does the scale matter?

Think it through in three steps.

- `Var(q·k)` as `d_k` grows?
- Effect on the softmax output?
- Effect on the gradients?



Scaling keeps the softmax out of saturation

dot-product variance

$$\mathrm{Var}(q \cdot k) = d_k$$

softmax Jacobian

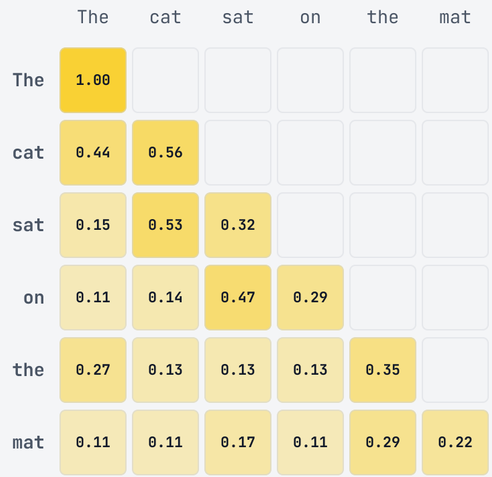
$$\frac{\partial s_i}{\partial z_j} = s_i \left(\delta_{ij} - s_j\right)$$

- Assumes independent, zero-mean, unit-variance coordinates of `q` and `k`
- Large `d_k` spreads the unscaled scores widely
- A nearly one-hot softmax passes almost no gradient
- Dividing by √d_k restores unit variance at initialization (Vaswani et al., 2017)

Masking, visualized

- Rows are queries, columns are keys
- With the mask off, weight flows to future keys
- The last row is unchanged: it already sees every key

ILLUSTRATIVE Options: scaling ON (÷√d_k, d_k = 16) causal mask ON



Rows: queries; columns: keys.

The causal mask hides future tokens

(4)

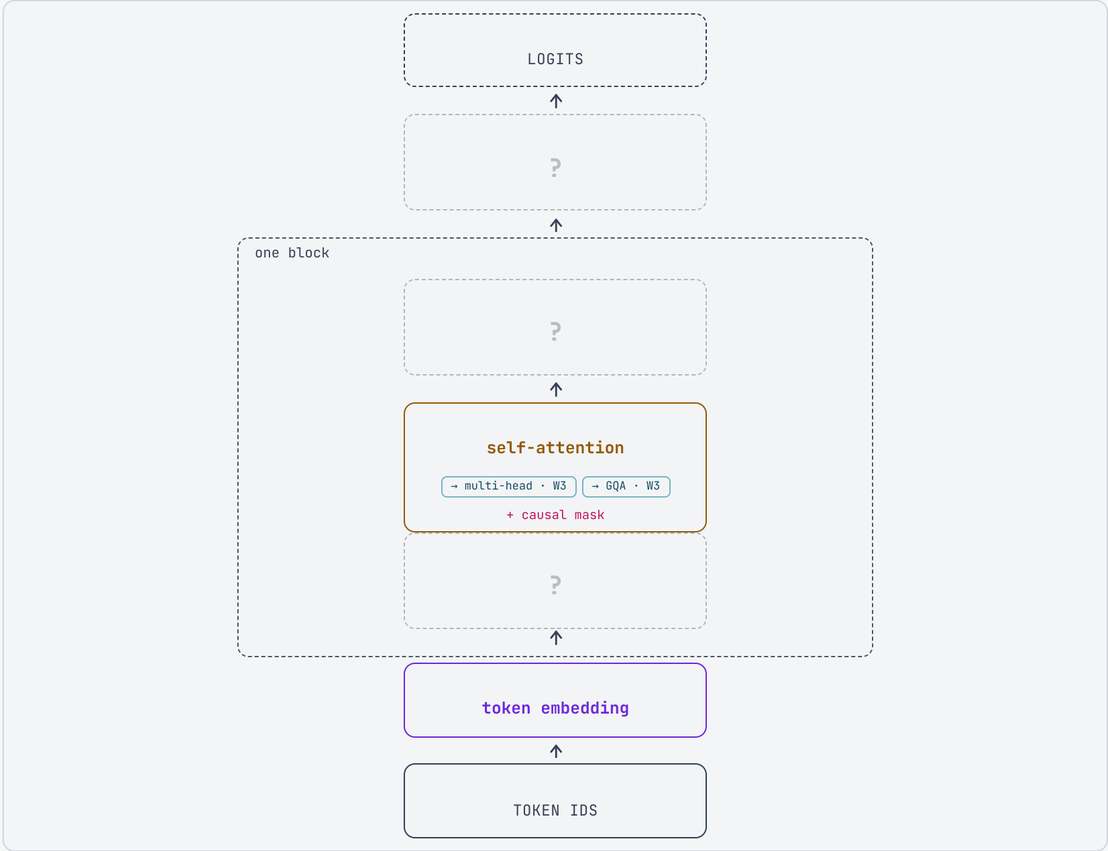
$$M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

- The diagonal is allowed: row `i` reads token `i`
- Row `i` predicts raw token `i+1`
- Added before the softmax, the mask gives future keys weight 0 (Vaswani et al., 2017)

Attention returns to width D

- The output projection `W_O` maps `[B,S,d_k]` back to `[B,S,D]`

Attention mixes positions



Without position, attention sees a set

- Permuting the inputs permutes the outputs the same way
- The mask adds an order boundary: each row sees a prefix (Haviv et al., 2022)
- The mask still gives no row a direct position vector

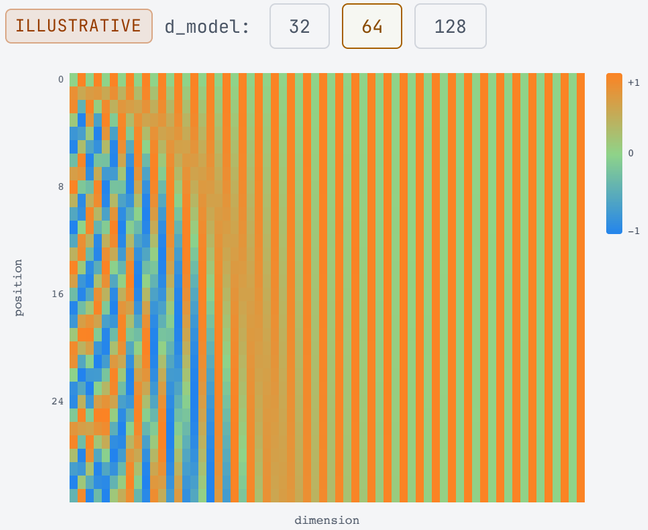


Sinusoids add position without parameters

(5)

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/D}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/D}}\right)$$

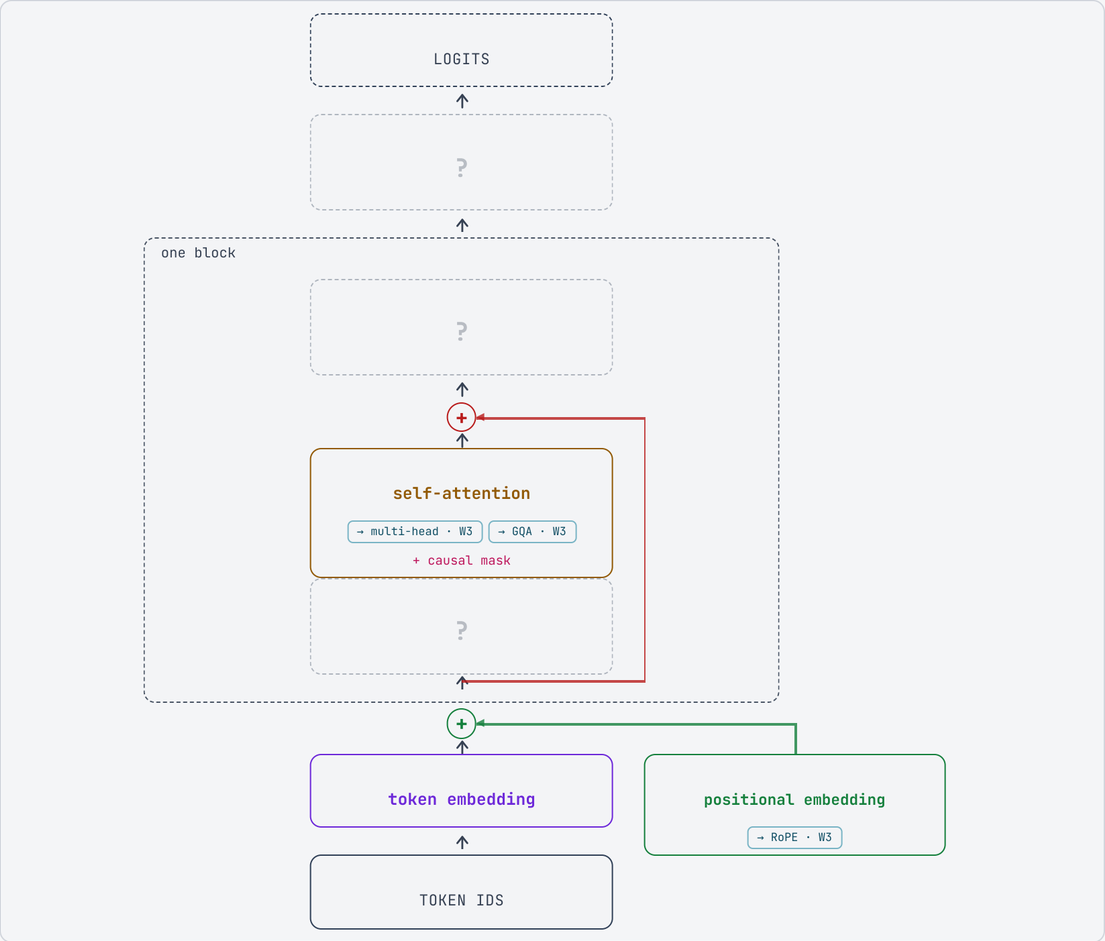
- Fixed: zero learned parameters (Vaswani et al., 2017)
- Dimension pairs run from fast to slow frequencies
- An offset of `k` positions rotates each pair



Residuals keep the input and add an update

$$x \leftarrow x + \mathrm{Sublayer}(\mathrm{RMSNorm}(x)) \tag{6}$$

- Branch learns an update: `x + F(x)`
- **Identity path** carries gradients
- Eases optimization, but does not guarantee stable gradients (He et al., 2016)



RMSNorm rescales each branch input

$$\mathrm{RMSNorm}(x) = g \odot \frac{x}{\sqrt{D^{-1}\sum_{i=1}^{D} x_i^2 + \epsilon}}, \qquad \epsilon = 10^{-5} \tag{7}$$

- **Root mean square normalization** (RMSNorm)
- No mean-centering, but a learned gain
- Pre-norm: better-behaved gradients at initialization (Xiong et al., 2020)
- The course still uses warmup

show: LayerNorm **RMSNorm**

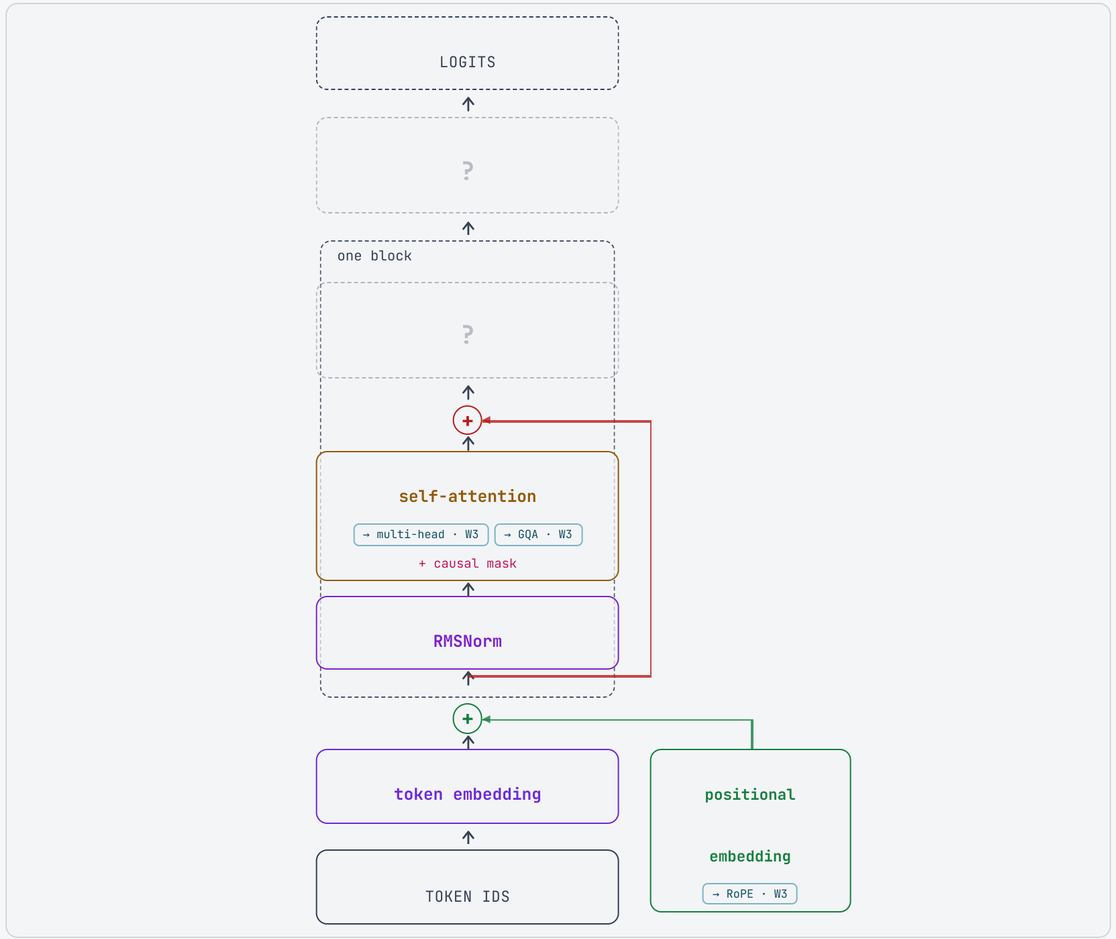
$$y = \gamma \odot \frac{x}{r}$$
$$r = \sqrt{\mathrm{mean}(x^2) + \epsilon}$$

RMSNorm drops: ~~mean-centering (μ)~~ ~~bias (β)~~

Zhang & Sennrich 2019 (RMSNorm).

RMSNorm acts on the branch input only

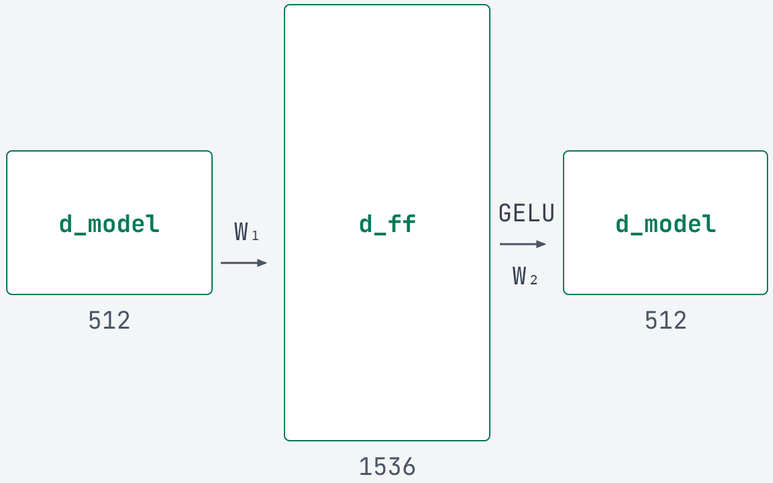
- Norm before attention and MLP
- The residual route bypasses the norm

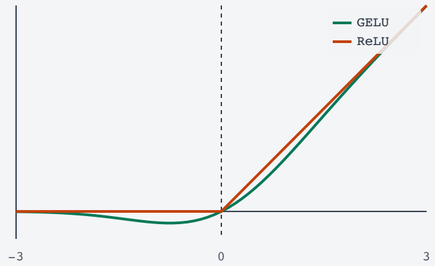


Attention mixes; the MLP transforms

$$\mathrm{MLP}(x) = \mathrm{GELU}(xW_1)W_2, \qquad D \to 3D \to D \tag{8}$$

- **MLP**: a nonlinearity per token
- `D → 3D → D`, GELU, no bias (Hendrycks & Gimpel, 2016)
- Same weights at every position





$$\mathrm{GELU}(x) = x\,\Phi(x)$$

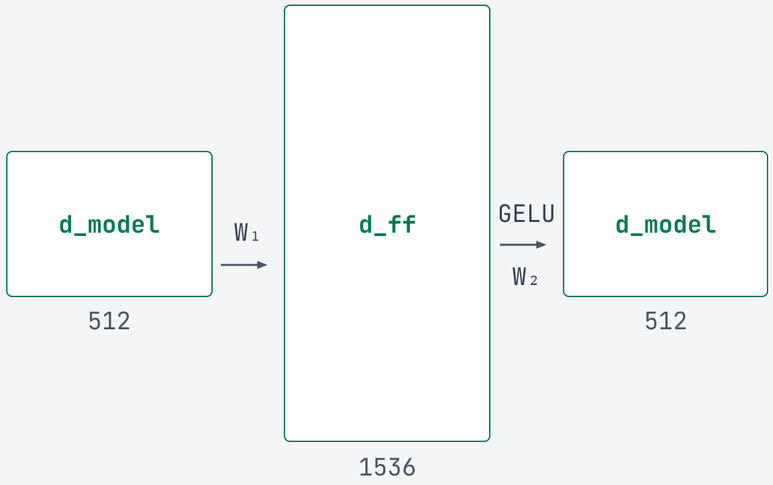
$$\Phi(x) = \tfrac{1}{2}\left[1 + \mathrm{erf}\left(x/\sqrt{2}\right)\right]$$

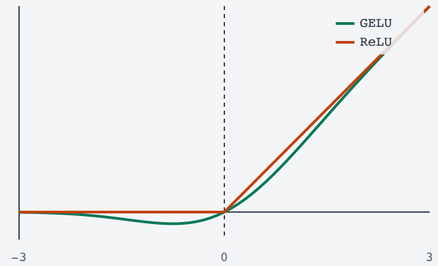
$$\mathrm{GELU}(x) \approx 0.5\,x\left(1 + \tanh\left[\sqrt{2/\pi}\,\left(x + 0.044715\,x^3\right)\right]\right)$$

Hendrycks & Gimpel 2016.

Attention or the MLP?

Which holds more of the block's parameters?





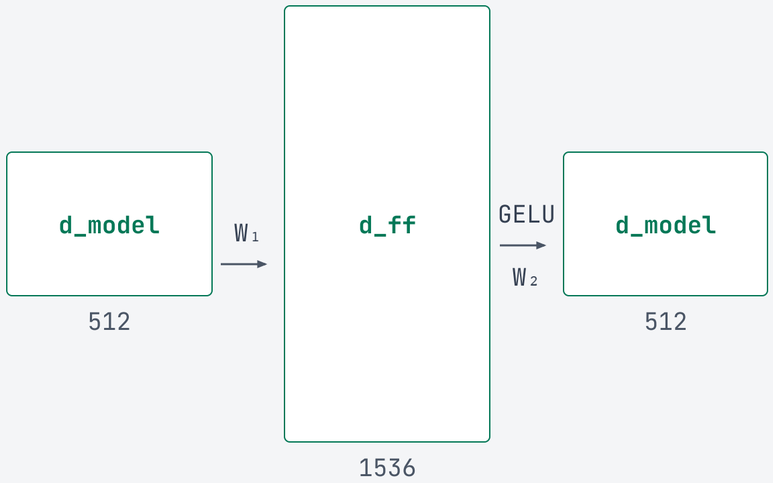
$$\mathrm{GELU}(x) = x\,\Phi(x)$$

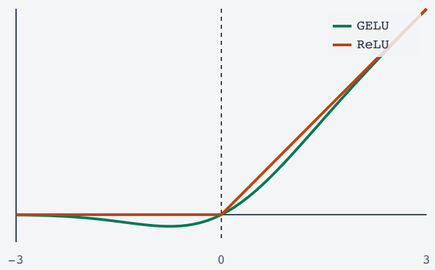
$$\Phi(x) = \tfrac{1}{2}\left[1 + \mathrm{erf}\left(x/\sqrt{2}\right)\right]$$

$$\mathrm{GELU}(x) \approx 0.5\,x\left(1 + \tanh\left[\sqrt{2/\pi}\,\left(x + 0.044715\,x^3\right)\right]\right)$$

Hendrycks & Gimpel 2016.

The MLP holds most of the block's weights





$$\mathrm{GELU}(x) = x\,\Phi(x)$$

$$\Phi(x) = \tfrac{1}{2}\left[1 + \mathrm{erf}\left(x/\sqrt{2}\right)\right]$$

$$\mathrm{GELU}(x) \approx 0.5\,x\left(1 + \tanh\left[\sqrt{2/\pi}\,\left(x + 0.044715\,x^3\right)\right]\right)$$

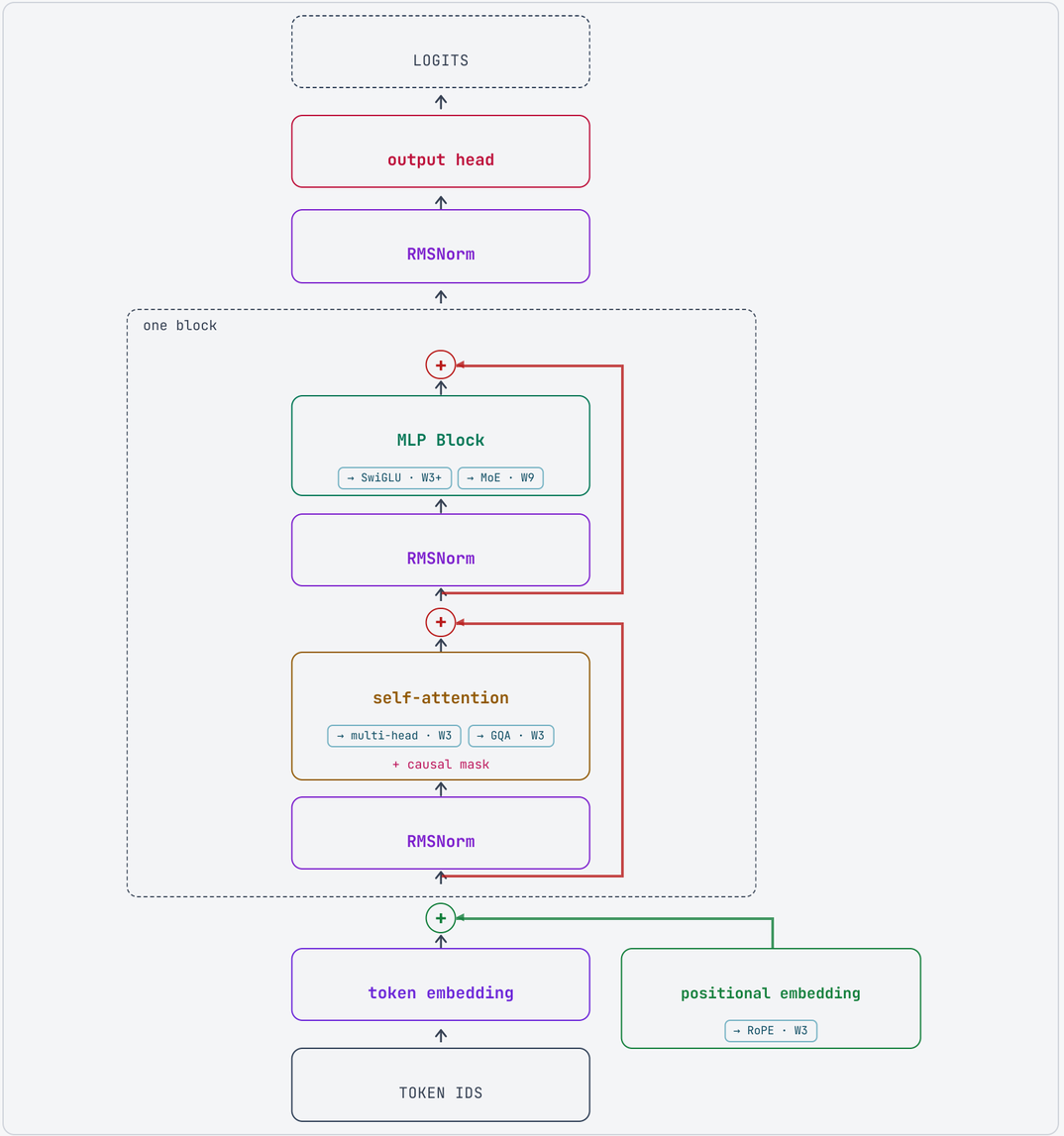
Hendrycks & Gimpel 2016.

- Attention: `4D² = 1,048,576` parameters
- MLP: `2·D·3D = 6D² = 1,572,864` parameters
- The MLP holds 60% of the block's matrix parameters

Four lines assemble the model

```
x = token_embedding(inputs) + sinusoidal_position
x = x + attention(rmsnorm_1(x), causal_mask)
x = x + mlp(rmsnorm_2(x))
logits = tied_head(final_rmsnorm(x))
```

- IDs `[B,S]` become states `[B,S,D]`, then logits `[B,S,V]`



Tying shares one table for input and output

(9)

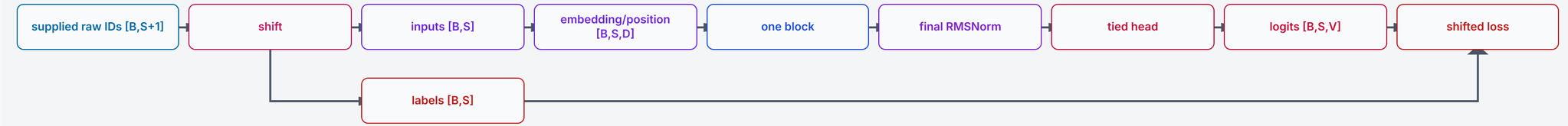
$$P_{tied} = VD + L(4D^2 + 2Dd_{ff}) + (2L + 1)D$$

Configuration	Parameters
tied, one block	`6,817,280`
untied: add `VD=4,194,304`	`11,011,584`

- **One table**: lookup at the input, scores at the output (Press & Wolf, 2017)
- Saves `VD = 4,194,304` parameters
- A course choice, not a rule

One window, from shift to loss

- Shifting raw `[B,S+1]` gives inputs and labels `[B,S]`
- Logit row `i` is scored against raw token `i+1`



Training repeats five steps

(10)

$$\mathcal{L} = -\frac{1}{BS}\sum_{b=1}^{B}\sum_{i=0}^{S-1}\log p_\theta(x_{b,i+1} \mid x_{b,\leq i})$$

1. Zero gradients
2. Forward to logits
3. Cross-entropy against shifted labels
4. Backward
5. Optimizer step and log

Five axes, five kinds of evidence

Question	Evidence
Quality	shifted cross-entropy loss
Compute	analytic matmul FLOPs; supported-op profiler estimate
Memory	analytic buckets; peak allocated and reserved bytes
Time	synchronized wall-clock tokens/s
Money	priced runtime outside the required exercise

Counting forward FLOPs

Component	Week 2 forward matmul FLOPs
Q, K, V, output projections	$8BSLD^2$
score and weighted-value matmuls	$4BS^2LD$
two MLP matmuls	$4BSLDd_{ff}$
tied vocabulary projection	$2BSVD$

- One multiply-add counts as 2 **FLOPs**
- Block count `L`, here 1
- Per token, the head needs 8.4M FLOPs, the block 7.3M

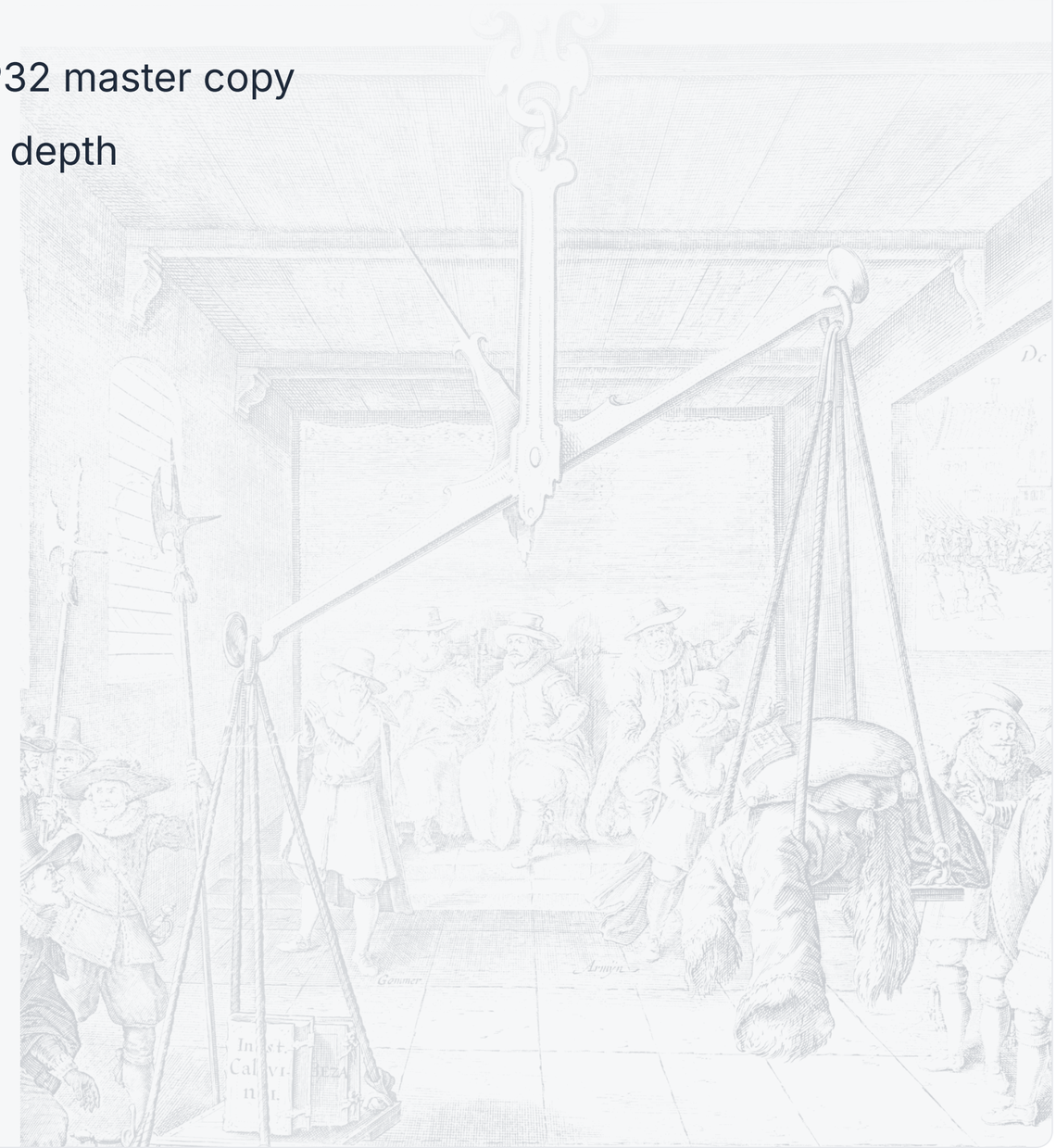
6NT is a coarse training estimate

(11)
$$C \approx 6NT \ (\text{forward} \approx 2NT,\ \text{backward} \approx 4NT)$$

- Forward ≈ `2NT`, backward ≈ `4NT`
- Training tokens `T`, width still `D`
- Omits attention's S^2 term (Hoffmann et al., 2022), norms, optimizer work and communication

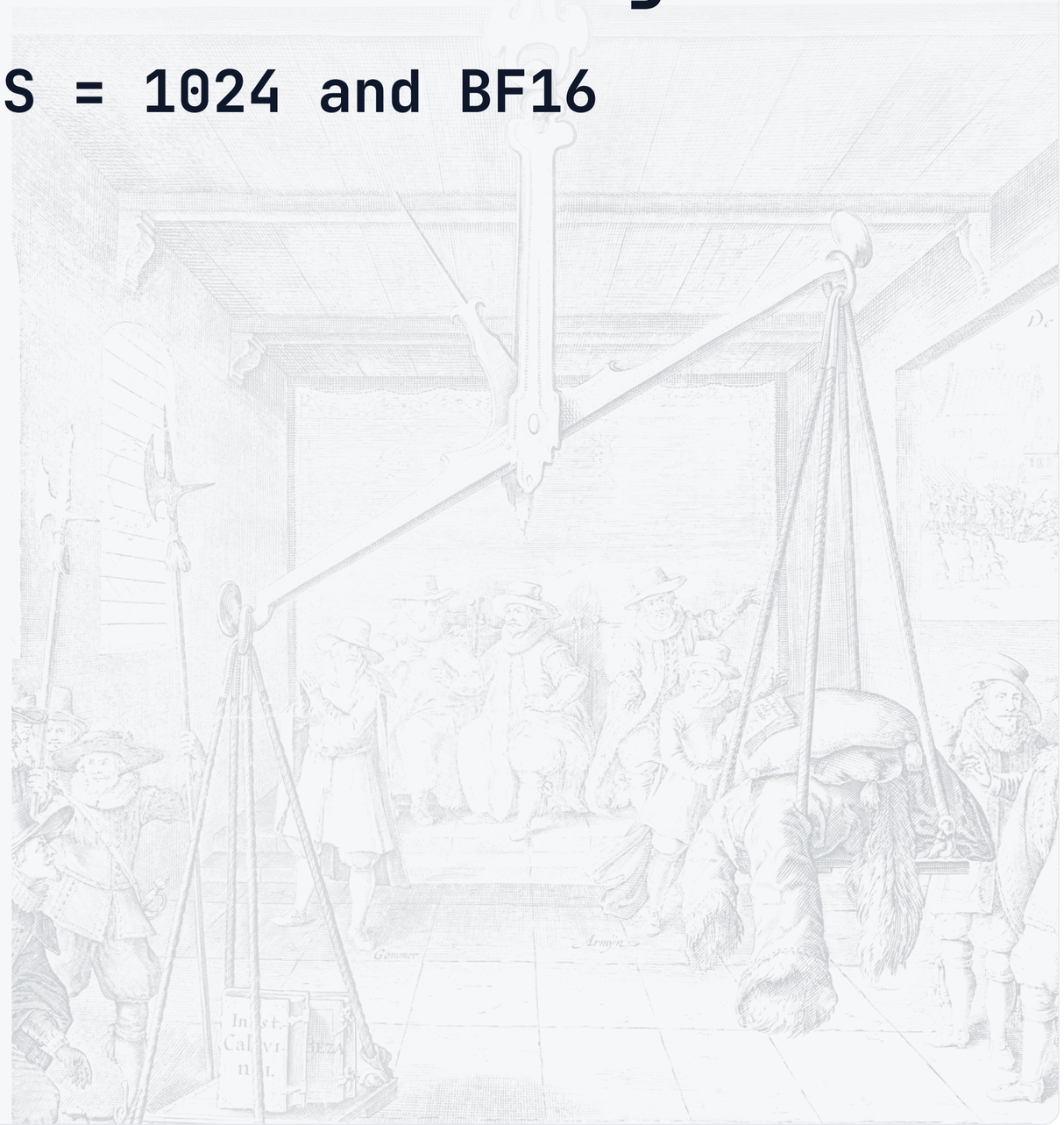
Training memory has seven buckets

- Parameters and gradients: P elements each
- Two FP32 Adam moments, plus an optional FP32 master copy
- Activations grow with batch, length, width and depth
- Attention matrices: $B \cdot L \cdot S^2$
- Logits: $B \cdot S \cdot V$

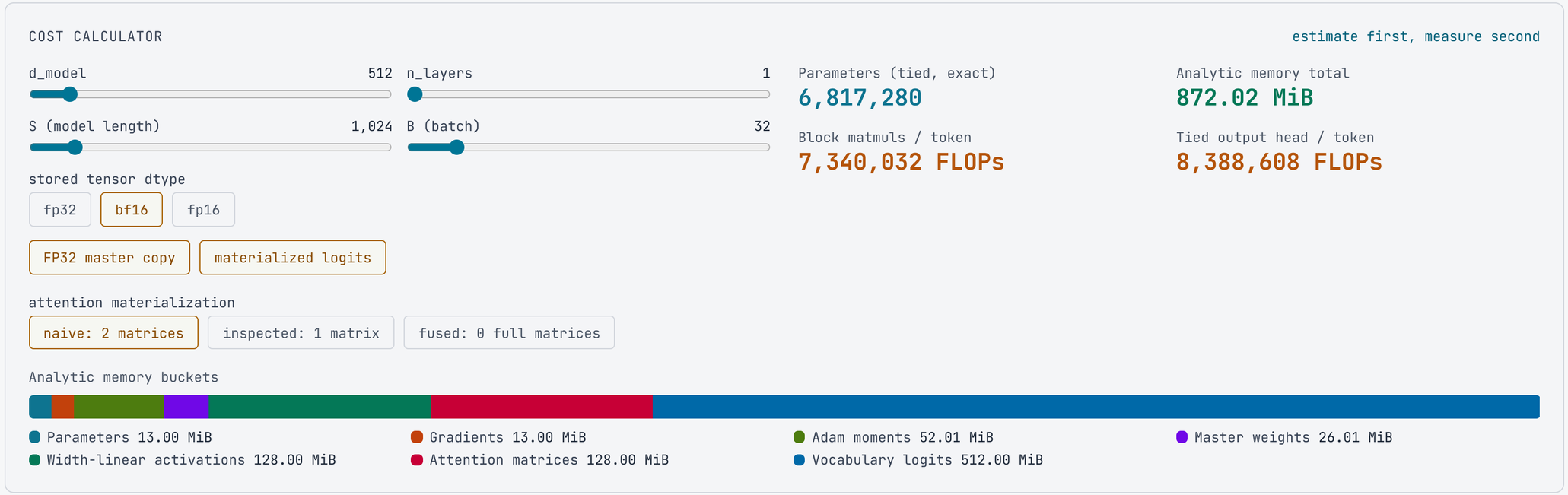


Which memory bucket is largest?

At B = 32, S = 1024 and BF16



At one block, the logits dominate memory



- At one block, the logits take 512 of 872 MiB
- At eight blocks, the logits take 512 MiB of 2.88 GiB
- Depth grows every bucket but the logits

How a model predicts the next token, and what it costs

1. `[B,S+1]` → `[B,S]` → logits `[B,S,V]`
2. Q, K, V `[B,S,d_k]`; scores `[B,S,S]`
3. ÷√d_k: unit variance at initialization
4. Mask: row `i` reads ≤ `i`, predicts `i+1`
5. `x + F(RMSNorm(x))`: attention, then MLP
6. Estimate, then measure: 6.8M, 872 MiB

To-dos until next week

Muddiest point

Do it right now! (2 min)

Before the tutorial

- Write down one question to discuss
- Think about the reflection questions

Self-study

- Read the reading slides and notes
- Explore the cited papers

Deadlines

- First exercise: Wednesday 28 October, 22:00
- Quiz 1: Friday 30 October

Reflection questions

1. Trace the tensor shapes and shifted labels through one training example. Where can an off-by-one bug hide?
2. What information comes from token embeddings, positional encoding, and the causal mask respectively?
3. Why do pre-normalization and residual paths make a deep transformer easier to optimize?
4. When can analytic FLOP estimates, profiler counts, and measured wall time disagree?
5. What does tying the input embedding and output head save, and what assumption does it make?
6. Which parts of this single-head block must change for multi-head attention, grouped-query attention, and rotary position embeddings next week?

